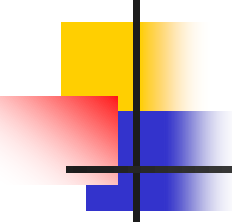


# The Unified Process

---

- The **Unified Process** (UP) is the software process attached to the UML [BRJ99,JBR99,Kru03].
- It employs **an incremental approach** to software development.
- The UP incorporates many elements of the Rational Unified Process (RUP) [Kru03].



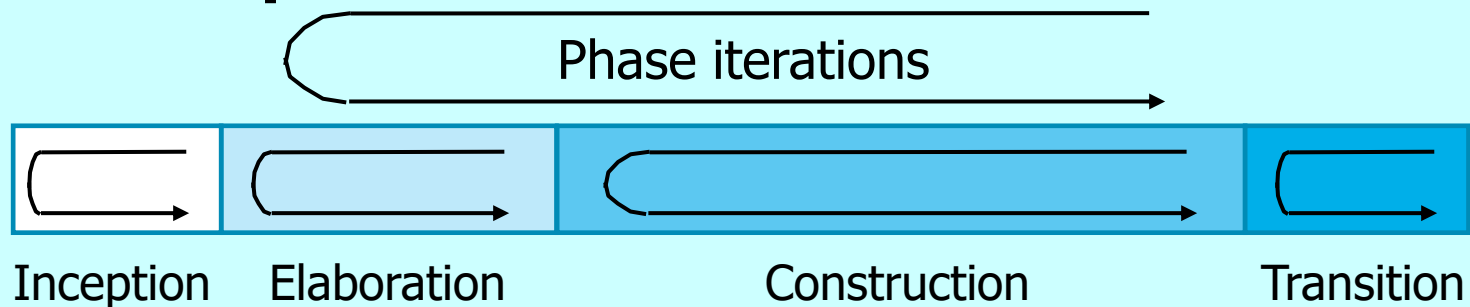
# The Unified Process

---

- The Unified Process [JBR99,Kru03]
  - is component-based,
  - uses the UML to create OOSE models, and
  - has three distinctive characteristics:
    - it is use-case driven,
    - it is architecture-centric,
    - it is iterative and incremental.

# The Unified Process

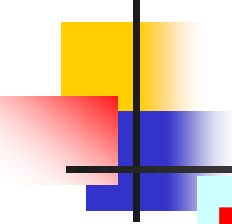
## Four phases in the UP



- Each phase consists of a number of **iterations**. Each iteration produces a new **increment**.
- Development is iterative and incremental.
- Most risks are addressed in inception and elaboration.
- Most resources are consumed in the construction phase.

In the UP an iteration consists of the following (basic / core) **workflows**:

- Requirements
- Analysis
- Design
- Implementation
- Test



# The Unified Process

- **Inception**

- Establish the initial business case for the system (costs and resources estimation).
- Most of the work in the iterations of the inception phase is accomplished in the requirements workflow.

- **Elaboration**

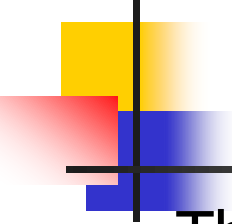
- Develop an understanding of the problem domain and the system architecture.
- The iterations of this phase involve all core workflows.

- **Construction**

- System design, implementation and testing.

- **Transition**

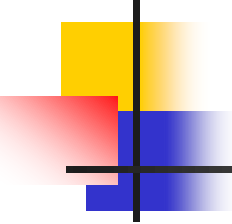
- Establish the system in its operating environment.
- The core workflows play a smaller role in this phase.



# The Unified Process

---

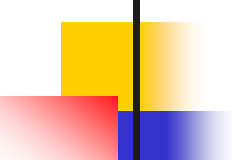
- The UP is use-case-driven.
  - Use-cases drive the software development process, from requirements capture to system testing.
  - A *use case* is a description of a set of sequences of actions, including variants, that a system performs that yields an observable result of value to a particular actor [BRJ99,JBR99].
- The UP is architecture centric.
  - In the UP, an architecture is a complete realization (from specification models to test models) of the architecturally relevant use cases (the architect decides what is *relevant*).
- The UP is iterative and incremental.
  - An increment is a realization of a set of use cases.
  - The approach is risk driven.



# Agile Methods

---

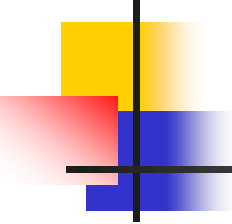
- **Agile methods** represent a recent evolution of the **incremental model** [Som07,Som11,Som16]
- Examples of agile approaches include: XP [Bec99], SCRUM [SB01], DSDM [Sta97], Crystal [Coc01] and FDD [PF02].
- Agile methods are best suited to small/medium-sized business systems or PC products [Som07, Som11,Som16].
  - However, if the software architecture provides mechanisms for compositional design, the success of small projects can be repeated for larger projects [FP16]
    - For example, using composable services in Service Oriented Architectures (SOA)



# Agile Methods

## Principles of agile methods

| Principle            | Description                                                                                                                                                                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer involvement | Customers should be closely involved throughout the development process. Their role is to provide and prioritize new system requirements and to evaluate the iterations of the system.                                                              |
| Incremental delivery | The software is developed in increments with the customer specifying the requirements to be included in each increment.                                                                                                                             |
| People not process   | The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working, without prescriptive processes. Agile methods emphasize real time (preferably face-to-face) communication. |
| Embrace change       | Expect the system requirements to change. Changing needs, rather than some meaningless inflexible plan, should guide the development process.                                                                                                       |
| Maintain simplicity  | Focus on simplicity. Wherever possible, actively work to eliminate complexity from the software system and from the development process. Agile methods emphasize working software, over written documentation.                                      |

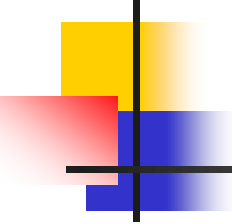


# Agile Methods

---

- Problems with agile methods:
  - It can be difficult to keep the interest of customers who are involved in the process.
  - Team members may be unsuited to the intense involvement (including face-to-face communication) that characterizes agile methods.
  - Prioritizing changes can be difficult where there are multiple stakeholders.
  - Maintaining simplicity requires extra work.
  - Contracts may be a problem as with other approaches to iterative development.

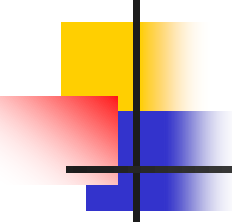




# Extreme Programming

---

- Extreme Programming (XP) is the best-known and most widely used agile method.
- XP takes an 'extreme' approach to iterative development.
  - Increments are delivered to customers every couple of weeks (in more conventional incremental approaches the development time per iteration is measured in months).
  - All tests must be run for every build and the build is only accepted if tests run successfully. Automated test cases are written before the software is developed.



# Extreme Programming

---

- XP values [Bec99]:

- **Communication**

- XP favors collaboration of clients and programmers. Ideally, (a representative of) the customer is full time engaged with the XP team. XP emphasizes frequent (preferably face-to-face) communication over written documentation.

- **Simplicity**

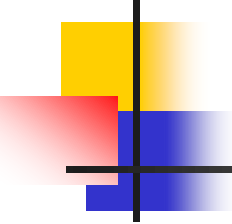
- XP encourages starting with the simplest solution and refactoring to better ones. However, XP realizes that achieving simplicity is not easy.

- **Feedback**

- Feedback is closely related with communication and simplicity. Programmers and the customer get feedback from the system through unit and acceptance testing. The customer gets feedback from the development team through direct estimates of the time that it will take to implement the new requirements.

- **Courage**

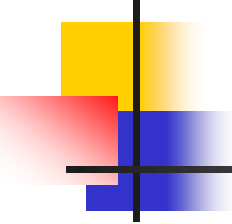
- Courage enables the developers to refactor their code, or even to throw away code and redesign the software architecture if the need arises.



# Extreme Programming

---

- In XP all requirements are expressed as **user stories** (scenarios).
  - **Scenarios** (**use cases** in the UP) are descriptions of example interaction sessions. Each scenario covers one or more possible interactions [Som05]. In practice, user stories can be more fragmented than traditional scenarios.
- User stories are written by the customer in English (or another natural language), using non-technical concepts.
  - Typically, a user story does not exceed 3-4 sentences.



# Extreme Programming

---

- User stories are the basis of project planning.
  - If the developers estimate that the implementation of a user story would take more than 2-3 weeks, the story must be broken down into smaller stories.
  - Based on business priorities and schedule estimates the customer decides which user stories will be developed in the next release (increment) of the software.
- XP relies on test-driven development (TDD)
  - Tests are developed from user stories before design and implementation
  - Acceptance (black box) tests are specified by the customer. Programmers continually write unit tests.
  - Automated test components are used to run all tests each time a new release is built.



# Extreme Programming

## XP practices

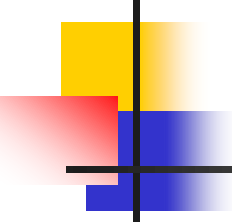
| Principle or practice  | Description                                                                                                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Incremental planning   | Requirements are recorded on Story Cards and the Stories to be included in a release are determined by the time available and their relative priority.                                 |
| Small releases         | The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release. |
| Simple design          | Enough design is carried out to meet the current requirements and no more.                                                                                                             |
| Test-first development | An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.                                              |
| Pair programming       | Developers work in pairs, checking each other's work and providing the support to always do a good job.                                                                                |
| Collective ownership   | The pairs of developers work on all areas of the system. No islands of expertise develop and all the developers own all the code. Anyone can change anything.                          |



# Extreme Programming

## XP practices

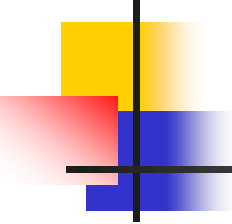
| Principle or practice  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Refactoring            | All developers are expected to <b>refactor</b> the code continuously as soon as possible code improvements are found. Examples of refactoring include: adding a Façade design pattern around a package to reduce coupling, adding a superclass that contains common features of existing classes, replacing a conditional statement with polymorphism, or even changing the name of a variable (or function) into something more meaningful. Refactoring should keep the code simple and maintainable. |
| Continuous integration | As soon as work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.                                                                                                                                                                                                                                                                                                                                                 |
| On-site customer       | A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an XP process, the customer is a member of the development team and is responsible for bringing system requirements to the development team for the implementation.                                                                                                                                                                                                         |
| Sustainable pace       | Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium-term productivity                                                                                                                                                                                                                                                                                                                                                                 |



# Extreme Programming

---

- XP replaces the traditional design for anticipated change with constant refactoring.
  - Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
  - In XP this is not considered worthwhile as changes cannot be reliably anticipated !
  - Rather, XP proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.

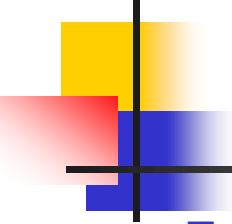


# Extreme Programming

---

- **Test-driven development** is one of the most important innovations in XP [Som07,Som16].
- In [LL05] this approach is described as follows:
  - In test-driven development you first write an automated set of tests. Next, you design and write the code such that the tests will pass. The tests are basically a restating of the requirements in an automated, testable form. In agile test-first development, the tests may actually be the only way that the requirements are stated in detail. ... Writing the automated tests helps ensure the requirements are properly understood.



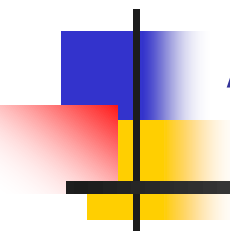


# Extreme Programming

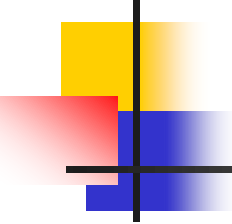
---

- **Pair programming** is another innovative XP practice.
  - In XP, programmers work in pairs, sitting together to develop code.
  - This helps develop common ownership of code and spreads knowledge across the team.
  - It serves as an informal review process as each line of code is looked at by more than one person.
  - It encourages refactoring as the whole team can benefit from this.
  - Measurements suggest that development productivity with pair programming is similar to that of two people working independently.

# Basic Software Development Activities



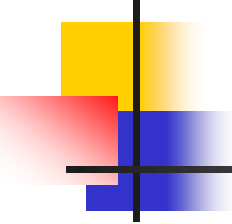
---



# Basic activities

---

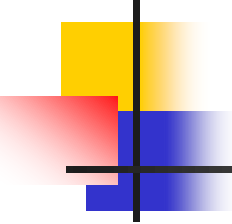
- In this chapter we discuss the basic software development activities:
  - specification,
  - design,
  - validation, and
  - evolution.
- The presentation is mainly adapted from [JBR99,Pre97,Pre00,Pre14,Som01,Som05,Som07, Som11,Som16].



# Basic activities – analysis and specification

---

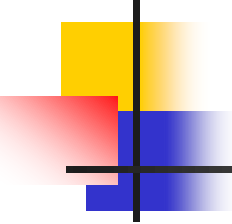
- In the **analysis** activity, the aim of the software developer is to establish the (functional and non-functional) software requirements.
- The requirements resulted from analysis must be documented and this activity is called **specification** [Som89,Som11]. Specification is a representation process [Pre97].
- These activities are often called **requirements engineering**.



# Basic activities – analysis and specification

---

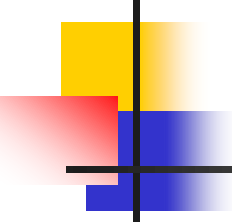
- **Analysis and specification principles** [Dav95] (cf. [Pre97]):
  1. The information domain of the problem must be represented and understood.
  2. The functions that the software is to perform must be defined. The behavior of the software (as a consequence of external events) must be represented.
  3. The functionality must be separated from the implementation. The analysis activity should produce a **cognitive model** rather than a design or implementation model. A cognitive model describes the system as perceived by its user community.
  4. The software specification should be tolerant of incompleteness, augmentable and **amenable to change**.



# Basic activities – analysis and specification

---

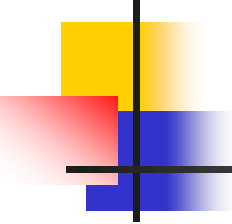
- Requirements fall into two main categories [Som89,Som11,Som16]:
  - **Functional requirements** These are the system services which are expected by the users of the system. Functional requirements may be validated by prototyping.
  - **Non-functional requirements** These set out the constraints under which the system must operate, the standards which must be met by the delivered system, etc.



# Basic activities – analysis and specification

---

- A software product provides
  - **functionality** (i.e. services), and
  - **non-functional attributes** which reflect the quality of that software.
- The **quality attributes** (the non-functional ones) reflect [Som01,Som11,Som16]
  - the behavior of the software while it is executing and
  - the structure and the organization of the software.
- Examples of **quality attributes** include: reliability, efficiency, maintainability, speed (response time), usability, safety, security, etc.



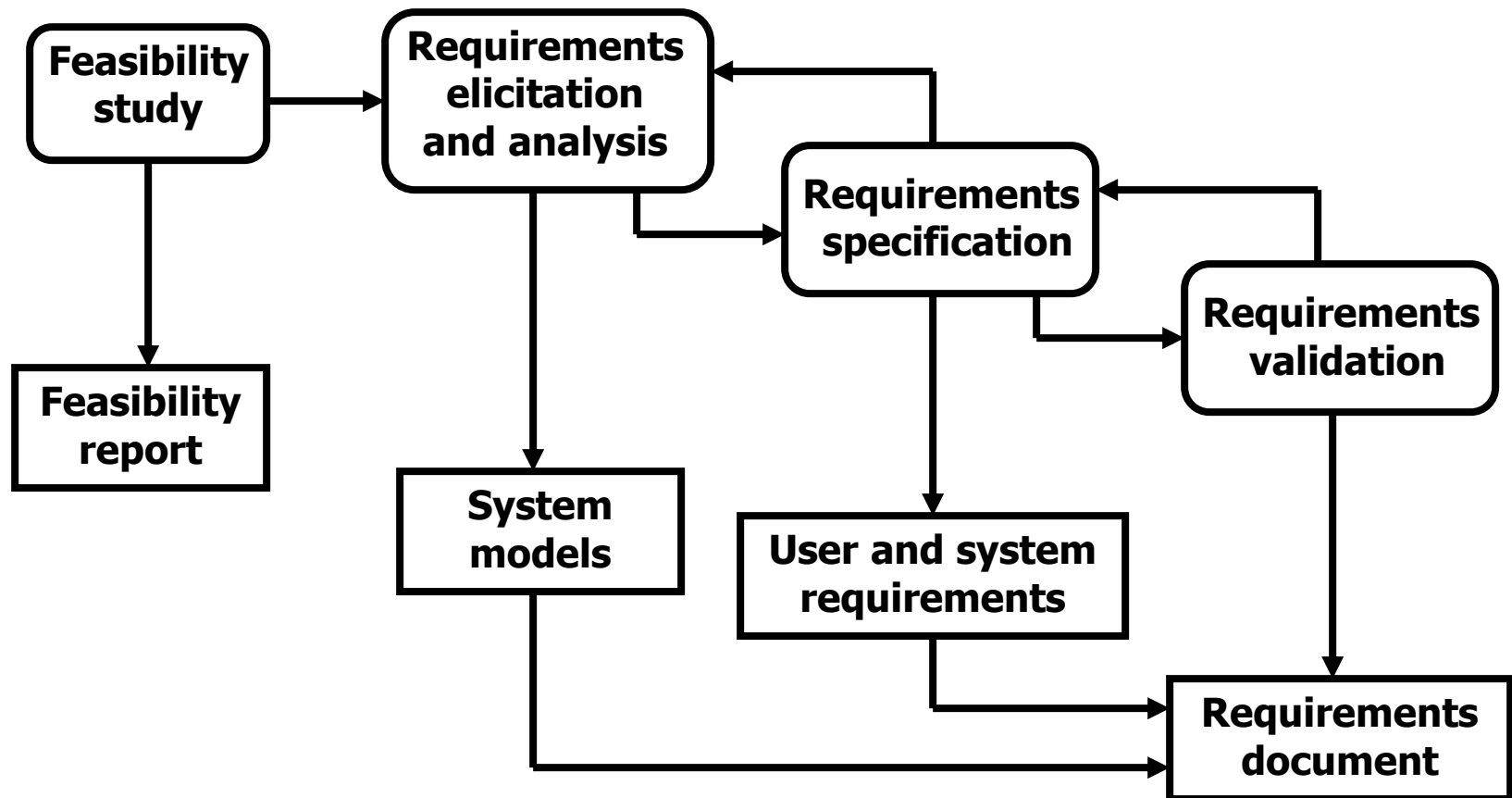
# Basic activities – analysis and specification

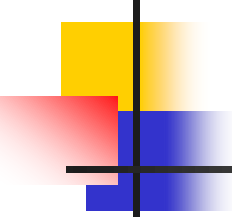
---

- Requirements engineering is a process that involves all of the activities required to create and maintain a **requirements document**.
- In large organizations, the structure of the requirements document may be standardized. There is an IEEE standard for the structure of the requirements document [IEEE93b].
- A **requirements engineering process** (leading to a requirements document) is given in the following figure [Som07,Som11]:



# Basic activities – analysis and specification

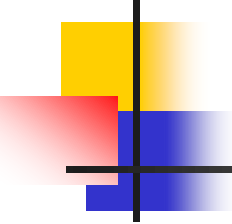




# Basic activities – analysis and specification

---

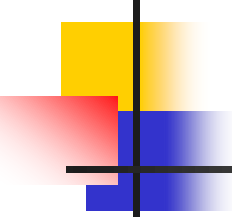
- Phases of the requirements engineering process [Som05,Som11]:
  - The feasibility study
    - decides whether the proposed system
      - can be cost effective (for the business of the customer),
      - can be developed using current technology and under the given budgetary constraints;
    - should be relatively cheap and quick.
  - Requirements elicitation and analysis
    - is the process of deriving requirements through
      - task analysis,
      - observation of existing systems, and
      - discussions with potential users;
    - it may involve the development of system models and prototypes.



# Basic activities – analysis and specification

---

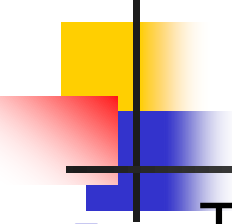
- Requirements specification
  - During this phase the information gathered during the analysis is translated into a requirements document.
    - Users and customers need a high-level statement of the requirements.
    - System developers need a more detailed system specification.
- Requirements validation
  - The requirements are checked for realism, consistency, and completeness.
  - Defects in the requirements document are inevitably discovered and corrected.



# Basic activities – analysis and specification

---

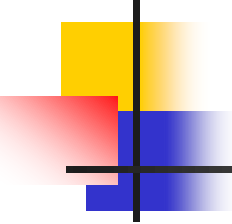
- Approaches to requirements elicitation and analysis:
  - *Structured analysis* [Mar79] (conventional SE)
  - *Prototyping*
  - *Scenarios* (**use-cases** in OOSE [Jac92,JBR99])
  - *Viewpoint analysis* This approach recognizes that a system can be perceived from different viewpoints, and uses viewpoints to structure the elicitation process and the requirements themselves. CORE [Mul79] and VORD [KS96] are methods that employ viewpoint analysis.
    - Ex. - major viewpoints for an ATM system: customer, auto-teller machine, account data base.
  - *Formal (i.e. mathematical) analysis and specification* (used only in specialized sub-domains)



# Basic activities – design

---

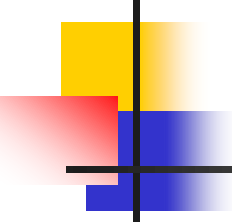
- The main input for software design is the requirements specification.
- **A specification is a cognitive model**, based on concepts taken from the domain of the problem, as perceived by the user community.
- **A design works as a blueprint for the implementation** [JBR99], employing concepts taken from the domain of the technical solution.
- The distinction between specification and design is sometimes expressed as follows:
  - **specification** represents **WHAT** to be implemented;
  - **design** is a representation of **HOW** to be implemented.



# Basic activities – design

---

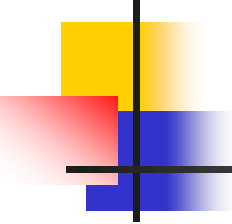
- **Design principles and guidelines** [Dav95] (cf. [Pre97]):
  1. A good designer should consider alternative approaches, judging each one based on the requirements of the problem, the resources available to do the job, and the design concepts.
  2. The design should not reinvent the wheel. Resources are limited. Design time should be invested in representing truly new ideas and in integrating those design patterns that already exist.
  3. The design should exhibit uniformity and integration.
  4. The design should be structured to accommodate change.
  5. The design should be traceable to the analysis model.
  6. The design should address behavioral and structural aspects from an implementation perspective.
  7. Design is not coding. The level of abstraction of the design model is higher than the source code.



# Basic activities – design

---

- 8. The design should “minimize the intellectual distance” between the software and the problem as it exists in the real world.
- 9. The design should lead to interfaces that reduce the complexity of the connections between modules and with the external environment.
- 10. The design should lead to modules that exhibit high cohesion and low coupling (ideally, each module should perform a single task, requiring little interaction with other modules).



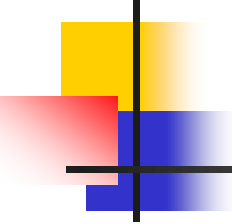
# Basic activities – design

---

## ■ **Design concepts:**

- *Architecture* - refers to the overall structure of the software system and the way in which that structure provides semantic integrity for the system [SG96].
  - In some approaches, only some sub-systems are considered relevant for the architectural view of a system [JBR99].
  - Software architecture relies on software *modularity*.
- A *module* can be seen as the basic building block which is used in modularization of software systems. In general, a module is not an independent system.
- A *sub-system* is a system in its own right.
  - A (sub-) system is composed of modules.





# Basic activities – design

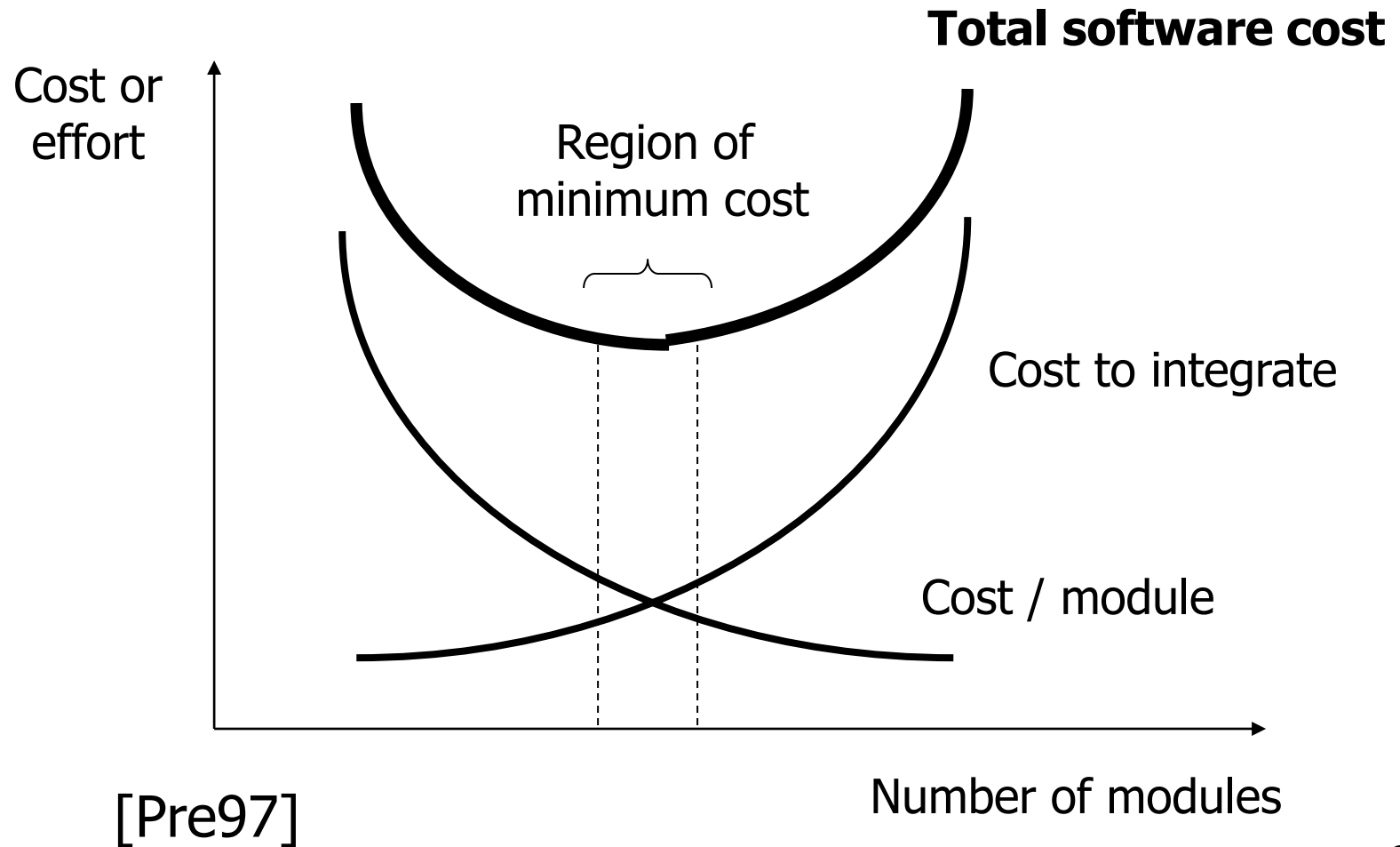
---

- The following ‘law’ - uncovered through experimentation in human problem solving – is an argument for *modularity* [Pre97]:

$$**E(p1 + ... + pn) > E(p1) + ... + E(pn)**$$

- $p1, ..., pn$  is a decomposition of a problem  $p$  into sub-problems
  - $E(x)$  is the effort required to solve problem  $x$
- In practice both *under-modularity* and *over-modularity* should be avoided.

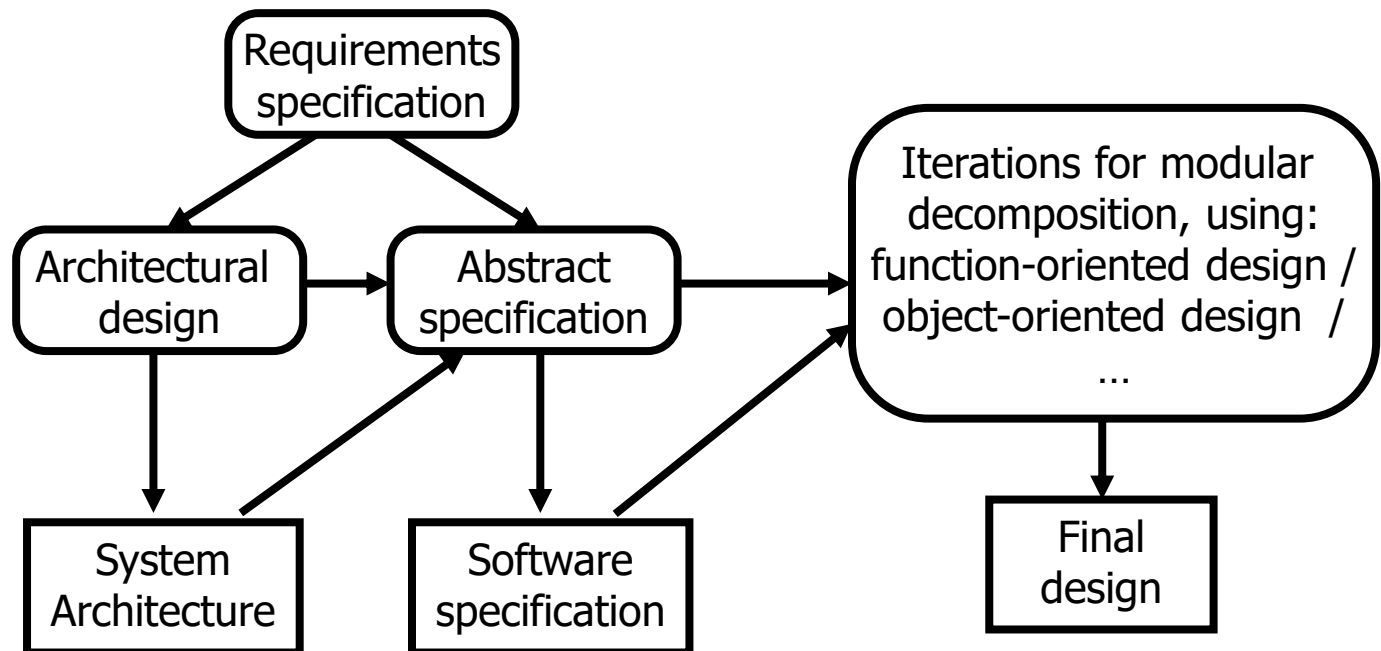
# Basic activities – design

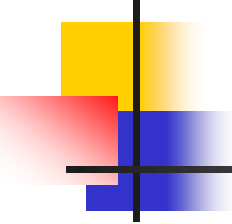


# Basic activities – design

- The design activity requires experience and creativity.
- A final design is an iteration from a number of preliminary designs.
- The design activity is a process of adding formality and detail (as the design progresses), with constant backtracking to correct earlier designs.

Here is a very general model of the **design process**:

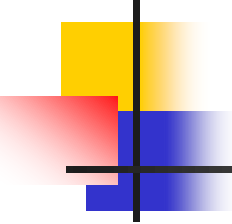




# Basic activities – design

---

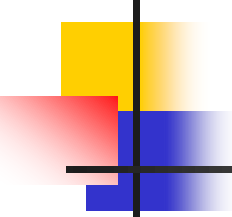
- The phases of the design activity [Som16,Som11,Som07,Som89]:
  - *Architectural design* The system is decomposed into a set of (related) sub-systems with their relationships. The output of this design process is a description of the *system architecture*.
  - *Abstract specification* For each sub-system, the abstract specification of its services and of the constraints under which it must operate is produced.
  - *Modular decomposition* In this phase, an appropriate design method (e.g. function-oriented, or object-oriented design) is employed, and after a number of iterations a final design is produced.



# Basic activities – design

---

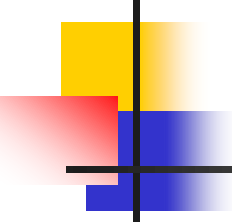
- There are two main kinds of models for the modular decomposition of sub-systems:
  - *Data-flow models* (for function-oriented design)
  - *Object-oriented models* (for object-oriented design)
- Basic strategies for modular decomposition:
  - ***Top-down design*** This strategy is based on the idea that the structure of the problem should determine the structure of the software solution.
  - ***Bottom-up design*** In practice, the top-down strategy is combined with the bottom-up strategy (designers find means for reuse as the top-down refinement progresses).



# Basic activities – software testing

---

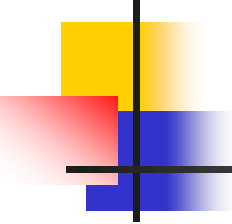
- In the **test** workflow, the aim of the software engineer is to verify and validate (V & V) the implementation.
  - All development activities – from specification to program development – involve V & V activities, such as inspections and reviews.
  - The most important V & V activity is software testing, which is performed after the implementation.
  - The difference between verification and validation [Boe79]:
    - **Verification** involves checking that the software system conforms to its specification.
      - Verification: are we building the product right?
    - **Validation** involves checking that the software product, as implemented, meets the expectations of the user.
      - Validation: are we building the right product?



# Basic activities – software testing

---

- Software testing is an important resource consuming phase:
  - 30%-40% in an ordinary project;
  - up to 5 times the resources of all other development activities in a project with stringent reliability, safety or security requirements (cf. [Pre97]).
- In projects based on formal methods unit testing can be replaced with a formal verification of the mathematical specification. This happens before the implementation.
  - But formal methods are employed only in specialized domains.
- In most projects, the most expensive V & V activity is performed after the implementation, when the system is tested.

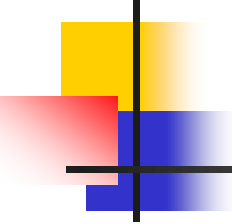


# Basic activities – software testing

---

- Two basic complementary approaches :
  - *Functional or black-box testing*
    - The system is treated as a 'black-box'. No information concerning the internal structure of the software is used.
    - Only the functional (input-output) behavior of the software is tested, using information from the software requirements.
  - *White-box (or glass-box) testing*
    - This is a way of testing software by using knowledge of its internal structure.
    - It relies on information gathered at design and implementation time.

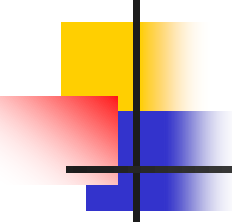




# Basic activities – software testing

---

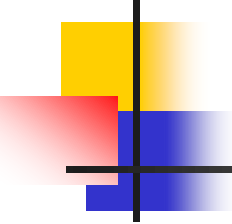
- Testing objectives [Mye79]:
  - Testing is a process of executing a program with the intent of finding a defect.
  - A good test case is one that has a high probability of finding an as-yet undiscovered defect.
  - A successful test is one that uncovers an as-yet undiscovered defect.
- However, we emphasize that [Dij72]
  - program testing can only demonstrate the presence of defects in a program, it can not demonstrate their absence!



# Basic activities – software testing

---

- Testing principles [Dav95] (cf. [Pre97]):
  1. All tests should be traceable to customer requirements (because the most severe defects are those that cause the program to fail to meet its requirements).
  2. Tests should be planned long before the testing begins. Black-box tests can be planned based on the requirements model. Detailed white-box test cases can be planned based on the design model.
  3. Testing should begin “in the small” (unit, module testing) and progress towards testing “in the large” (integration and user testing).
  4. Exhaustive testing is not possible.
  5. To be most effective, testing should be conducted by an independent third party.



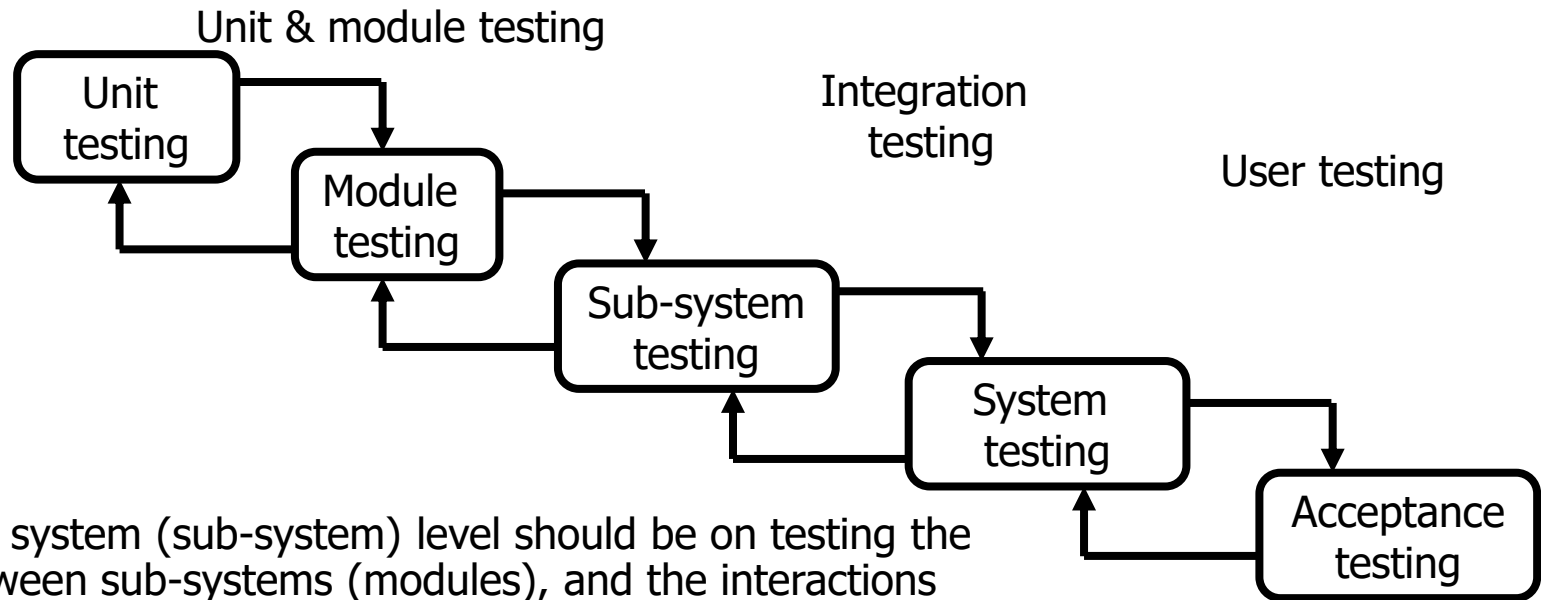
# Basic activities – software testing

---

- Software testing involves the following stages:
  - unit (e.g. procedure or function) testing,
  - module (e.g. class, ADT, or file module) testing,
  - sub-system testing,
  - system testing, and
  - acceptance testing (based on customer supplied data; sometimes called *alpha testing*).
  - (A *beta testing* stage can follow, when the system is delivered to a number of potential customers for further experiments.)

# Basic activities – software testing

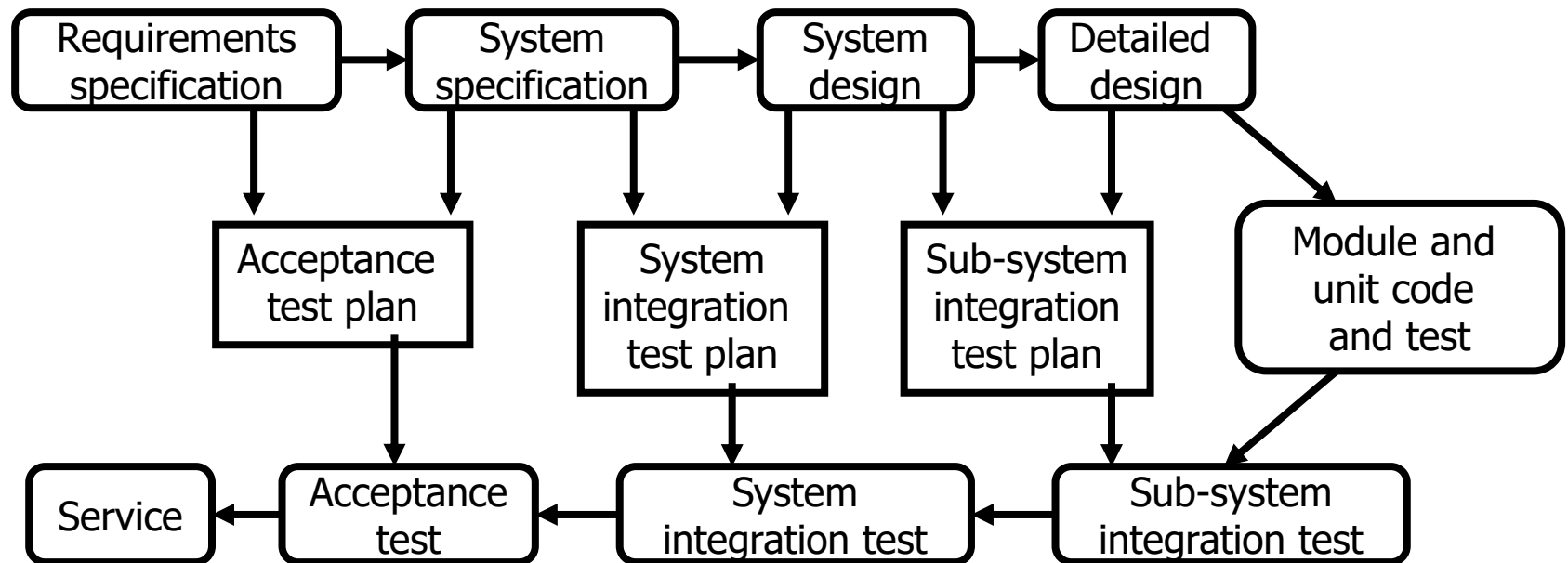
- The testing process:



- The focus at system (sub-system) level should be on testing the interfaces between sub-systems (modules), and the interactions between sub-systems (modules).
- During acceptance testing the system is tested with customer supplied data (rather than simulated test data). Both functional and non-functional requirements can be validated at this stage.

# Basic activities – software testing

- The relationship between development and testing activities through test plans:

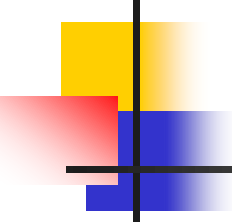




# Basic activities – software evolution

---

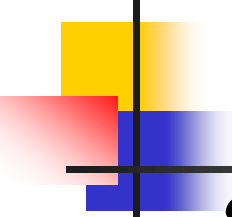
- Most software systems must evolve in order to meet changing or new requirements.
  - The traditional demarcation between software development and software evolutions is becoming increasingly irrelevant [Som11].
    - Few software systems are now completely new systems.
    - New technologies emerged, in particular reuse-oriented software engineering.
  - Evolution costs can be higher than the corresponding software development costs. However, software technology can provide important benefits [Som05,Som16]:
    - Software systems are more flexible than hardware (and other engineering) systems.
    - Software change costs are lower than the costs of corresponding changes to system hardware.
    - An increasing amount of software components are incorporated in various engineering systems.



# Basic activities – software evolution

---

- Software change strategies (applicable to legacy systems) include [Som01,Som05,Som11,Som16]:
  - *Software maintenance* The main types of software maintenance are:
    - defects repair,
    - adapt software to new operating environments,
    - modify or add functionality.
  - *Architectural evolution* (e.g. the transformation of a legacy system centralized architecture to a distributed client-server architecture)



# Basic activities – software evolution

---

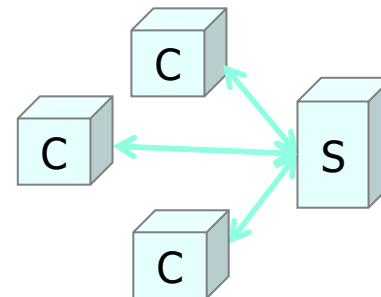
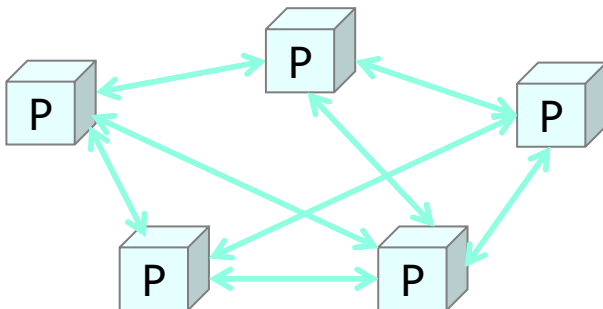
## ■ *Software re-engineering*

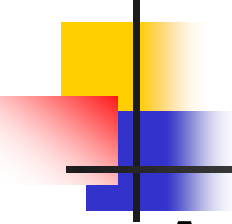
- The software system is modified to make it easier to understand and change.
- No new functionality is added and, normally, the software architecture is not changed.
- Changes may include:
  - re-documenting the system,
  - reorganizing and restructuring the system,
  - translating the implementation to a new programming language,
- CASE tools support is important or even essential for the re-engineering process. CASE tools may provide support for forward engineering, reverse engineering, automatic source code translation, etc.



# Specification and Architecture

- Turning back to the starting point of a project
  - Which comes first: software specification or software architecture? There is no simple answer
    - Specification and architecture are developed together
    - Often the best solution is obtained by iterating [JBR99]
- Architectural patterns (e.g., Peer-to-Peer, Client-Server) provide general, reusable solutions

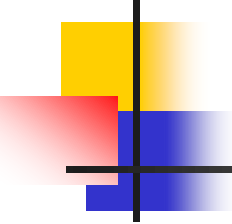




# Specification and Architecture

---

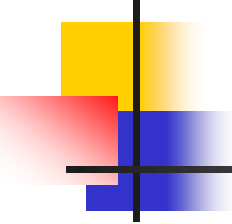
- **Agile** approaches (e.g., XP) rely on [Som16,FP16]
  - Customer involvement, **user stories** and behavioral driven design (BDD), test driven development (TDD)
  - Frameworks and tools to accelerate development
  - There is a potential downside: an agile project may meet customer expectations but not result in a good software architecture [FP16]
- In the **plan-and-document** approach software architecture is built using [JBR99,LL05]
  - Architectural patterns
  - Architecturally relevant **use cases**



# References

---

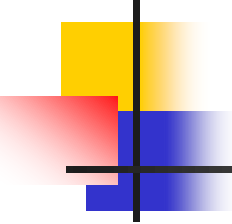
- [Bec99] K. Beck. Embracing Change with Extreme Programming. *IEEE Computer* 32(10):70-78, 1999.
- [Boe79] B. Boehm. Software engineering; R & D trends and defense needs. In *Research Directions in Software Technology*, pages 1-9, MIT Press, 1979.
- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [Coc01] A. Cockburn. *Agile software development*. Addison-Wesley, 2001.



# References

---

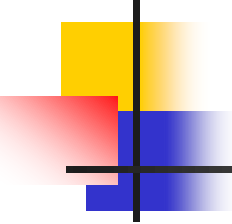
- [Dav95] A. Davis. *201 Principles of Software Development*. McGraw-Hill, 1995.
- [Dij72] E. Dijkstra, et al. *Structured programming*. London: Academic Press, 1972.
- [FP16] A. Fox, D. Patterson. *Engineering Software as a Service: An Agile Approach using Cloud Computing*. Strawberry Canyon Publisher, 2016.
- [IEEE93b] IEEE recommended practice for software requirements specifications. *Software Engineering Requirements Engineering*, R.H. Thayer, M. Dorfman, editors, Los Alamitos, CA: IEEE Computer Society Press.



# References

---

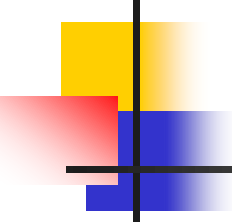
- [Jac92] I. Jacobson. *Object-Oriented Software Engineering*. Addison-Wesley, 1992.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [KS96] G. Katonya, I. Sommerville. Requirements engineering with viewpoints. *BCS/IEEE Software Engineering J.*, 11(1):5-18, 1996.
- [Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3<sup>rd</sup> edition). Addison-Wesley, 2003.
- [LL05] T. Lethbridge and R. Laganriere. *Object-Oriented Software Engineering* (2<sup>nd</sup> edition). McGraw-Hill, 2005.



# References

---

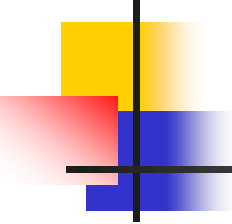
- [Mar79] T. DeMarco. *Structured Analysis and System Specification*. Prentice-Hall, 1979.
- [Mul79] G. Mullery. CORE – a method for controlled requirements specification. In *Proc. 4th Int. Conf. On Software Engineering*, IEEE Press, 1979.
- [Mye79] G. Myers. *The Art of Software Testing*. Wiley, 1979.
- [Pre97] R. Pressman. *Software Engineering: A Practitioner's Approach*, (4th edition). McGraw-Hill, 1997.
- [Pre00] R. Pressman. *Software Engineering: A Practitioner's Approach*, (5th edition). McGraw-Hill, 2000.
- [Pre14] R. Pressman. *Software Engineering: A Practitioner's Approach*, (8th edition). McGraw-Hill, 2014.



# References

---

- [SB01] K. Schwaber, M. Beedle. *Agile software development with SCRUM*. Prentice Hall, 2001.
- [SG96] M. Shaw and D. Garlan. *Software Architecture*. Prentice-Hall, 1996.
- [Som89] I. Sommerville. *Software Engineering*, (3rd edition). Addison-Wesley, 1989.
- [Som01] I. Sommerville. *Software Engineering*, (6th edition). Addison-Wesley, 2001.
- [Som05] I. Sommerville. *Software Engineering*, (7th edition). Addison-Wesley, 2005.



# References

---

- [Som07] I. Sommerville. *Software Engineering*, (8th edition). Addison-Wesley, 2007.
- [Som11] I. Sommerville. *Software Engineering*, (9th edition). Addison-Wesley, 2011.
- [Som16] I. Sommerville. *Software Engineering*, (10th edition). Addison-Wesley, 2016.
- [Sta97] J. Stapleton. *DSDM Dynamic Systems Development Method*. Addison-Wesley, 1997.